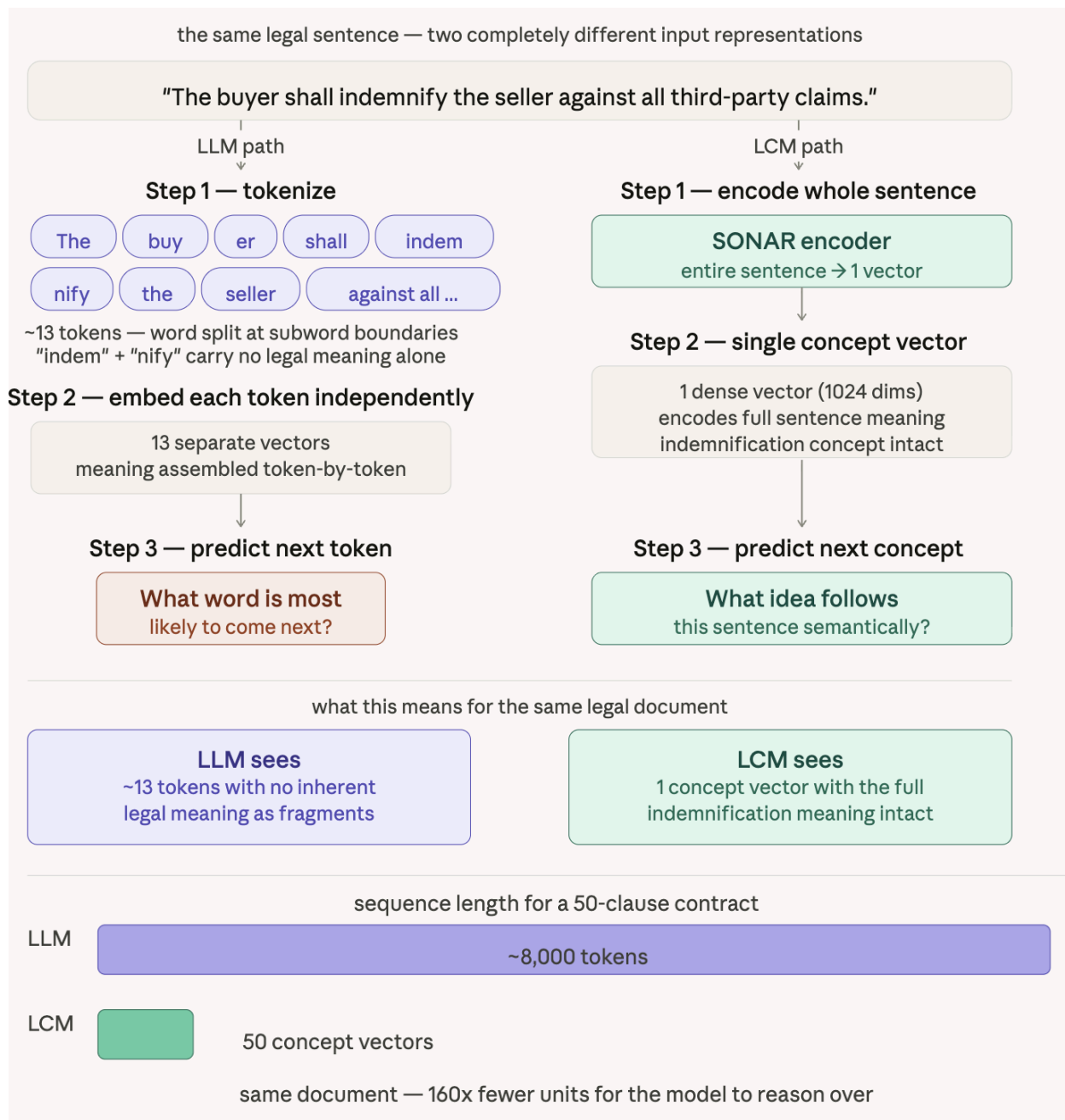


Key differences

Input unit

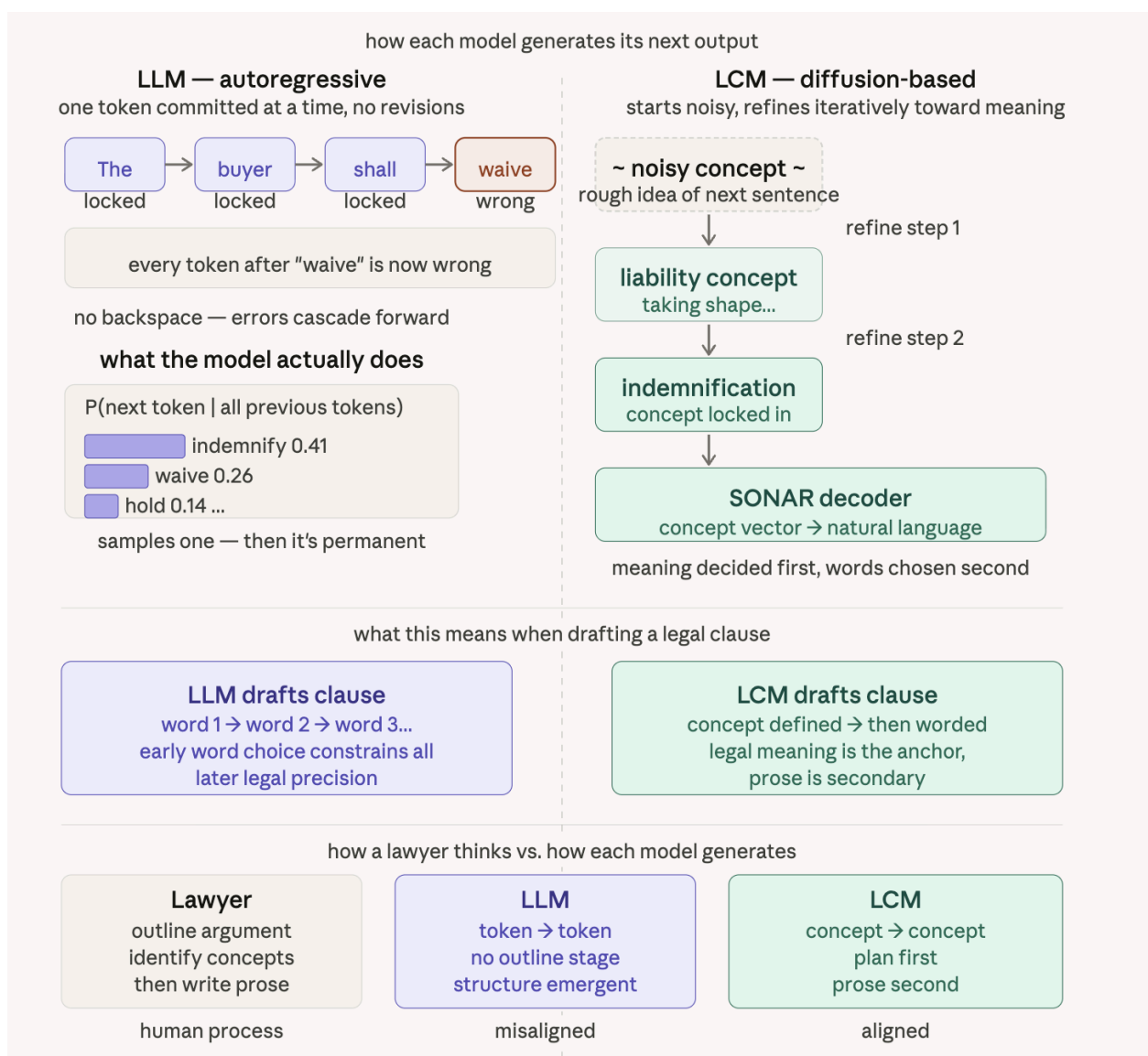


LLM: Most people assume an LLM reads words. It doesn't. It reads subword fragments produced by an algorithm called Byte Pair Encoding (BPE). The tokenizer splits text at statistically common boundaries, which means "indemnification" might become ["indem", "nif", "ication"] — three fragments that carry zero legal meaning independently. The model has to reconstruct the meaning of the word from these pieces using attention

across them, before it can even begin to think about what the word means in context. This is fine for common vocabulary, but legal language is full of precise technical terms — "indemnification," "tortfeasor," "estoppel," "subrogation" — that get fragmented in ways that obscure their meaning from the start.

LCM: LCM's input unit is the output of the SONAR encoder applied to an entire sentence. That encoder produces a single dense vector — typically around 1024 dimensions — where the coordinates in that high-dimensional space encode the complete semantic content of the sentence. Crucially, the encoding is holistic: the meaning of "indemnification" in context, its relationship to "buyer" and "seller," the directionality of obligation, all of it is compressed into that single vector simultaneously. Nothing is fragmented. The legal concept arrives at the model intact.

GENERATION MECHANISM



What autoregressive actually means: Every LLM generates text through the same mechanical process: given everything generated so far, compute a probability distribution over the entire vocabulary, sample one token, append it, repeat. The critical constraint is that each token is permanently committed the moment it's chosen. The model has no internal state that says "I'm generating an indemnification clause" — it only has the sequence of tokens it has already produced. This means the entire legal meaning of a sentence has to somehow emerge from a left-to-right chain of local token decisions, each one blind to where the sentence will eventually land.

In practice this causes a specific failure pattern lawyers would immediately recognize: the model begins a clause in a way that sounds reasonable, then paints itself into a corner — the early word choices constrain the legal precision of everything that follows. A clause that starts "The buyer agrees to..." is now syntactically committed to a specific legal structure before the model has "decided" what the legal obligation actually is

.

What diffusion-based generation actually means: Diffusion generation works by starting with a noisy, undefined concept vector and iteratively denoising it — running it through a series of refinement steps until it converges on a clear, coherent point in SONAR semantic space. Think of it like developing a photograph in a darkroom: the image starts as pure noise, gradually the outlines emerge, then the detail sharpens, and only at the end do you have a clear picture. The model is asking "what concept should come next?" at every refinement step, not "what word comes next?" — and it can revise its answer at each step before committing.

Only once the concept vector is fully refined does the SONAR decoder convert it into natural language. This means the legal meaning is determined before a single word is written.

How models train (penalty)(hallucinations)



What each model internalizes over training

This is the deepest consequence. An LLM trained on billions of legal documents learns that "indemnify" tends to follow "shall," that "whereas" tends to open recitals, that

"notwithstanding" signals an exception. It learns the statistical texture of legal language. An LCM trained on the same documents learns that obligation concepts tend to be followed by remedy concepts, that definition clauses tend to precede operative clauses, that exception language tends to sit adjacent to liability-limiting ideas. It learns the conceptual grammar of legal documents — the logic of how ideas relate — not just the surface patterns.

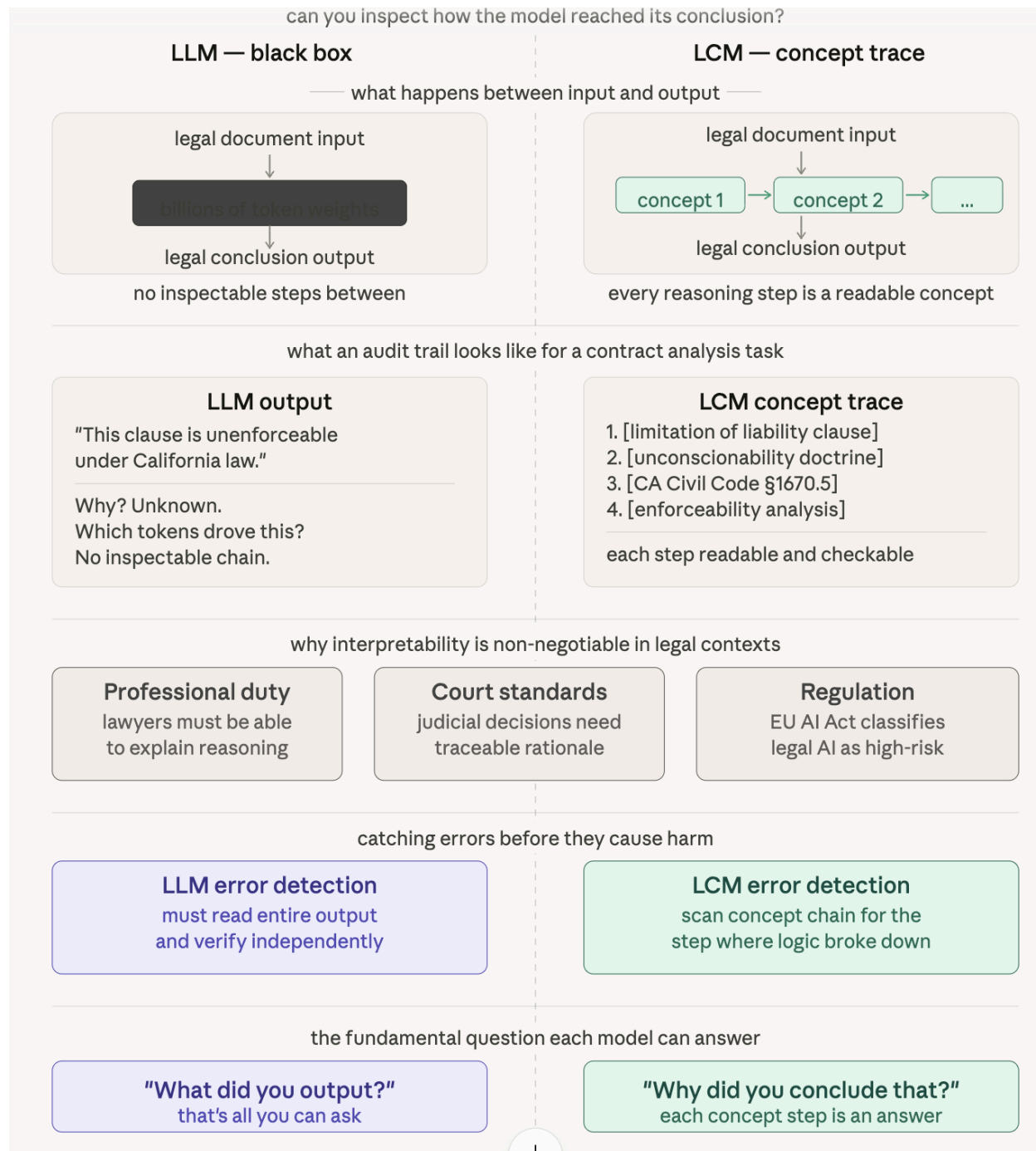
For your paper this distinction has a very concrete implication. When you test both models on novel legal documents — contracts from unfamiliar jurisdictions, new statute types, emerging regulatory frameworks — the LLM's statistical patterns may not transfer well because the surface language is different. The LCM's conceptual relationships should transfer much better because the underlying legal logic is the same even when the words are different. That's a directly testable hypothesis.

Why this is the root cause of hallucination

Legal AI hallucination — fabricated case citations, invented statutes, plausible-sounding but wrong holdings — all trace back to this training objective mismatch. The LLM is not "lying" when it generates a fake citation. It's doing exactly what it was trained to do: generating the token sequence that is statistically most plausible given the context. A realistic-looking case citation is statistically plausible in legal text, so the model produces one regardless of whether it exists. There is no signal in cross-entropy loss that says "this citation doesn't refer to a real case." LCMs don't eliminate this problem entirely, but because the training signal is anchored to actual semantic content rather than surface plausibility, the model is at least being rewarded for conceptual accuracy — which is a fundamentally closer proxy to legal correctness than token prediction ever can be.

In one sentence for your paper: LLMs are trained to sound like legal experts, LCMs are trained to think like them.

Reasoning Transparency



When an LLM makes a legal error, detecting it requires a human expert to read the entire output and independently verify every claim — you have no shortcut to where the reasoning went wrong. When an LCM makes an error, you can scan the concept chain

and identify the specific step where the logic broke down. Was the error in identifying the relevant doctrine? In applying it to the facts? In drawing the conclusion? The concept trace tells you precisely where the model went off track, which makes human oversight dramatically more efficient. In a legal workflow where a lawyer is reviewing AI output before it goes to a client or a court, this difference in review efficiency is enormous.

Three pressures converge on legal AI that don't exist in most other domains. First, professional responsibility rules in most jurisdictions require lawyers to supervise and understand any work product they put their name to — a lawyer cannot ethically rely on an AI conclusion they cannot explain. Second, courts increasingly scrutinize AI-generated legal arguments, and several high-profile sanctions have already been issued against lawyers who submitted AI-hallucinated citations without verification. Third, the EU AI Act explicitly classifies AI systems used in legal proceedings as high-risk, imposing transparency and auditability requirements that black-box LLMs structurally cannot meet. LCMs' concept traces are architecturally positioned to satisfy these requirements in a way LLMs simply are not.

Sources claudes used: **What came from the original Meta research paper**

The core technical details — SONAR embeddings, the concept vector approach, diffusion-based generation, MSE loss over embeddings, the 200+ language coverage, the sequence length reduction — all of that comes directly from Meta AI's foundational LCM paper published in December 2024. The full citation is:

"Large Concept Models: Language Modeling in a Sentence Representation Space" — Meta AI, December 2024. It's available on arXiv at arxiv.org/abs/2412.08821.

That paper is the primary source for everything structural about how LCMs work. It should be your first and most-cited technical reference.

What came from the web searches I did earlier in our conversation

When you first asked me about LCMs vs LLMs, I ran searches and pulled from secondary sources — articles from DataCamp, Medium, GeeksforGeeks, InfoQ, and a few others — which I cited inline at the time. Those were useful for framing and accessible explanations but they are not peer-reviewed and should not be cited directly in an academic paper. They helped me build the explanations, but you'd want to trace every technical claim back to the original Meta paper.

What is my own synthesis and extension

The legal-specific arguments — how the input unit maps onto clause-level legal drafting, the professional responsibility framing, the EU AI Act angle, the cascading error problem, the audit trail argument — those are my own analytical extensions connecting the LCM architecture to the legal domain. They are reasoned arguments grounded in the technical facts, but they aren't directly sourced from a paper. That's actually where your original research contribution lives — no paper has made these connections yet, which is precisely why your capstone is viable.